miniweather-heterogenous-port

Authors:

Prabhsharan Singh

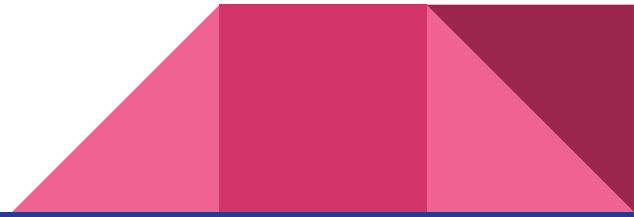
Emilio Gordillo

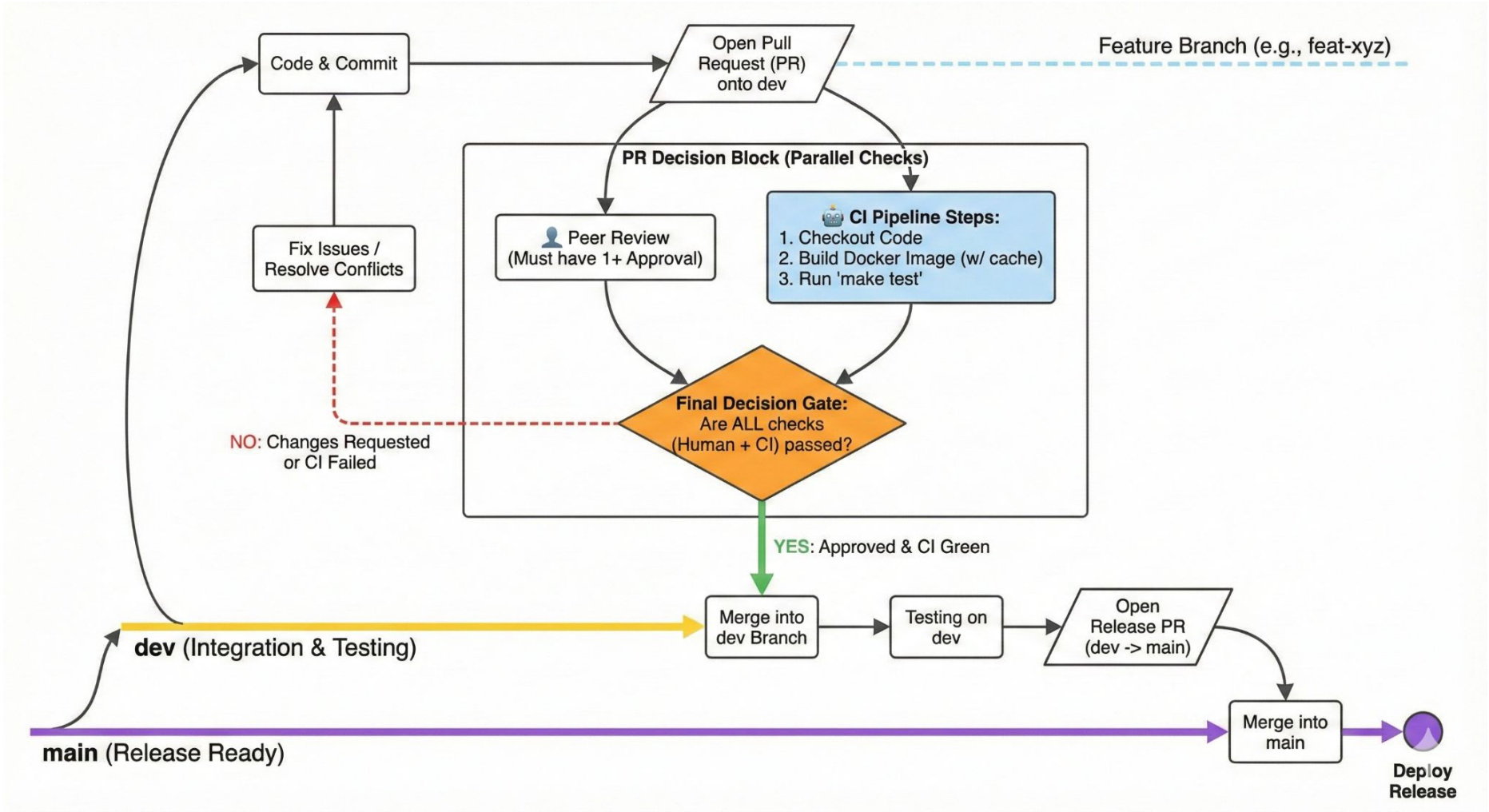
J. Franco Ray

SOW for the project

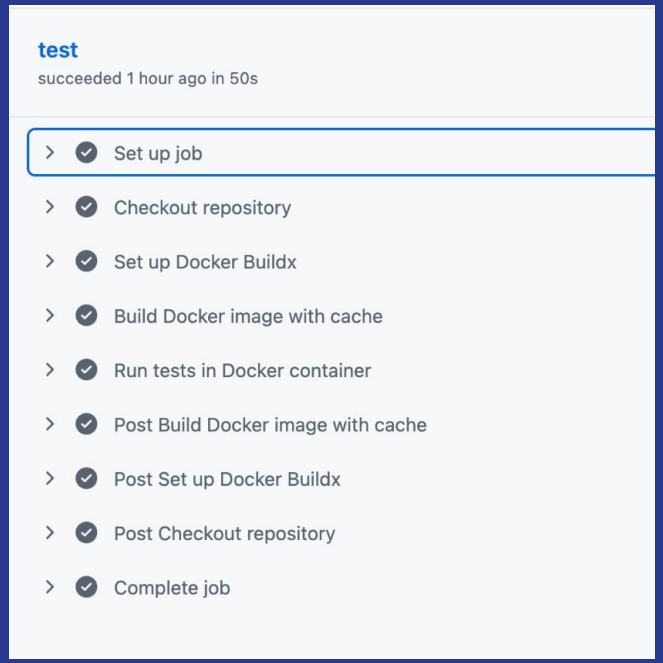
- Understand the source code
- Accept input parameters in command line
- Multithreading using OMP
- Distributed computation+multithreading using MPI+OMP
- Distributed computation+GPU offloading using MPI+ACC
- Parallel output file generation
- Documentation
- CMake with automated testing
- Containerization and continuous integration

Development Workflow with CI-CD




Code & Commit
Open Pull Request (PR) onto dev
Feature Branch (e.g., feat-xyz)
PR Decision Block (Parallel Checks)
Peer Review (Must have 1+ Approval)
CI Pipeline Steps:
1. Checkout Code
2. Build Docker Image (w/ cache)
3. Run 'make test'
Fix Issues / Resolve Conflicts
Final Decision Gate: Are ALL checks (Human + CI) passed?
NO: Changes Requested or CI Failed
YES: Approved & CI Green
dev (Integration & Testing)
Merge into dev Branch
Testing on dev
Open Release PR (dev -> main)
main (Release Ready)
Merge into main
Deploy Release

Github Actions




```yaml
name: CI

on:
  push:
    branches: [ dev, main ]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-22.04

    permissions:
      contents: read

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Build Docker image with cache
        uses: docker/build-push-action@v6
        with:
          context: .
          tags: atmosphere-model:latest
          load: true            # load the image into the local Docker daemon
          push: false           # we're not pushing to a registry
          cache-from: type=gha  # use GitHub Actions cache as buildx cache backend
          cache-to: type=gha,mode=max

      - name: Run tests in Docker container
        run: |
          docker run --rm \
            -v "${{ github.workspace }}/code/serial:/app" \
            atmosphere-model:latest \
            make test
```

Fix multi gpu #49

Resolving conflicts between `fix-multi-gpu` and `dev` and committing changes → `fix-multi-gpu`

1 conflicting file

module_types.F90
code/serial/module_types.F90

code/serial/module_types.F90

```
101
    Accept current change | Accept incoming change | Accept both changes
102 <<<<<<< fix-multi-gpu (Current change)
103   subroutine init_halo_buffers()
104   implicit none
105
106   if (halos_allocated) return
107
108   halo_size = hs * nz * NVARS
109   allocate(send_left(hs, nz, NVARS), send_right(hs, nz, NVARS))
110   allocate(recv_left(hs, nz, NVARS), recv_right(hs, nz, NVARS))
111 #ifdef USE_OPENACC
112   ! Keep halo buffers resident on the device to avoid per-call staging
113   !$acc enter data create(send_left, send_right, recv_left, recv_right)
114 #endif
115   halos_allocated = .true.
116   end subroutine init_halo_buffers
117
118   subroutine free_halo_buffers()
119   implicit none
120   if (.not. halos_allocated) return
121 #ifdef USE_OPENACC
122   if (acc_is_present(send_left)) then
123     !$acc exit data delete(send_left, send_right, recv_left, recv_right)
124   end if
125 #endif
126   if (allocated(send_left)) deallocate(send_left)
127   if (allocated(send_right)) deallocate(send_right)
128   if (allocated(recv_left)) deallocate(recv_left)
129   if (allocated(recv_right)) deallocate(recv_right)
130   halo_size = 0
131   halos_allocated = .false.
132   end subroutine free_halo_buffers
133
134 =======
135   !> @brief Allocate state storage with halos and set component pointers.
136 >>>>>>> dev (Incoming change)
137   subroutine new_state(atmo)
```

Testing suite: make test

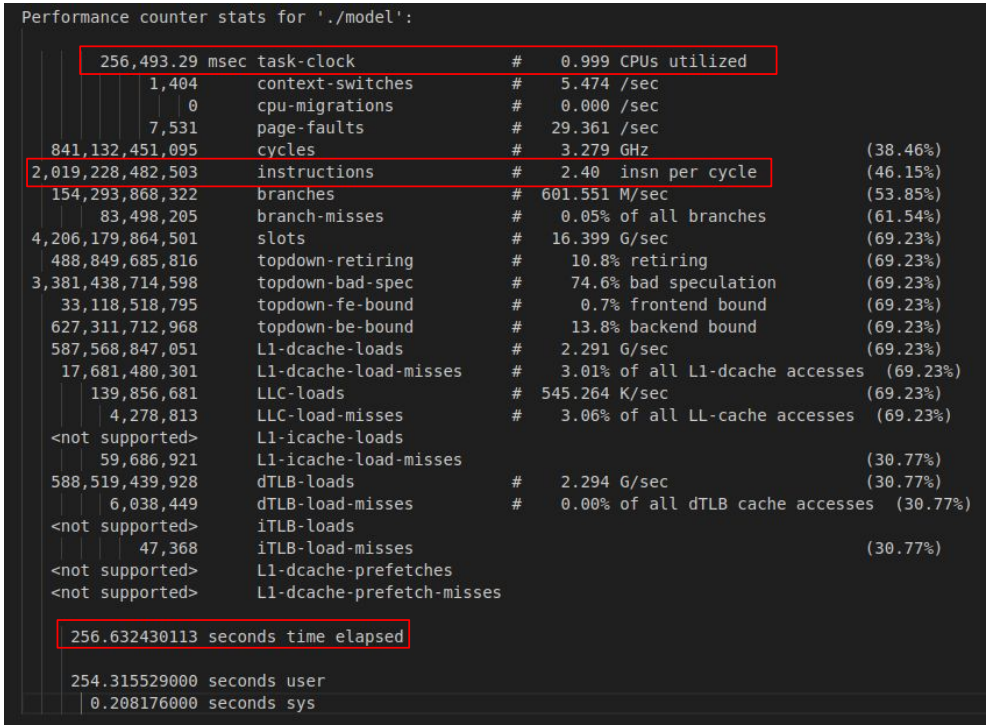
```
test: model
    @echo "TEST: Running model..."
    @OUTPUT=$$(mpirun -np 3 ./model 100 1000 10); \
#   echo "$$OUTPUT"; \
    MASS=$$(echo "$$OUTPUT" | awk '/Fractional Delta Mass/{print $$NF}'); \
    ENERGY=$$(echo "$$OUTPUT" | awk '/Fractional Delta Energy/{print $$NF}'); \
#   echo "Fractional Delta Mass   = $$MASS"; \
#   echo "Fractional Delta Energy = $$ENERGY"; \
    awk -v m="$$MASS" -v e="$$ENERGY" 'BEGIN { \
      mass = m + 0; energy = e + 0; \
      if (mass < 1e-13 && energy < 1e-3) { \
        print "TEST PASSED."; exit 0; \
      } else { \
        print "TEST FAILED: thresholds exceeded."; \
        printf "  mass   = %e  (limit 1e-13)\n", mass; \
        printf "  energy = %e  (limit 1e-3)\n", energy; \
        exit 1; \
      } \
    }'
    @if [ -f output.nc ]; then \
      echo "TEST: Compare output with reference output file..."; \
      python3 nccmp3.py output-serial.nc output-serial-optimized.nc output.nc; \
    else \
      echo "Output file not found, skipping comparison test."; \
    fi
```

CMake

```cmake
cmake_minimum_required(VERSION 3.18)
project(AtmosModel LANGUAGES Fortran)

# Option to control OpenACC vs OpenMP build (like USE_OPEN
option(USE_OPENACC "Enable OpenACC (NVHPC) instead of Oper

# ------------------------------------------------------------
# NetCDF (Fortran + C) flags via nf-config / nc-config
# ------------------------------------------------------------
execute_process(
    COMMAND nf-config --fflags
    OUTPUT_VARIABLE NF_FFLAGS
    OUTPUT_STRIP_TRAILING_WHITESPACE
)
execute_process(
    COMMAND nf-config --flibs
    OUTPUT_VARIABLE NF_FLIBS
    OUTPUT_STRIP_TRAILING_WHITESPACE
)
execute_process(
    COMMAND nc-config --libdir
    OUTPUT_VARIABLE NC_LIBDIR
    OUTPUT_STRIP_TRAILING_WHITESPACE
)

# Convert the strings to CMake lists
separate_arguments(NF_FFLAGS)
separate_arguments(NF_FLIBS)

# ------------------------------------------------------------
# MPI
# ------------------------------------------------------------
find_package(MPI REQUIRED Fortran)

# Base NetCDF flags
target_compile_options(model PRIVATE ${NF_FFLAGS})

# Choose between OpenACC and OpenMP-style build
if(USE_OPENACC)
    message(STATUS "Building with OpenACC (NVHPC)")
    target_compile_definitions(model PRIVATE USE_OPENACC)
    target_compile_options(model PRIVATE
        -fast
        -Minfo=accel
        -Mpreprocess
    )

    # Link flags: need -acc at link time, optionally gpu flags
    target_link_options(model PRIVATE
        -acc
        -gpu=cc80
    )
else()
    message(STATUS "Building with OpenMP-style flags")
    target_compile_definitions(model PRIVATE USE_OPENMP)
    target_compile_options(model PRIVATE
        -Ofast
        -fPIC
        -fopenmp
        -cpp
    )

    # Optional: also use CMake's OpenMP discovery if available
    find_package(OpenMP)
    if(OpenMP_Fortran_FOUND)
        target_link_libraries(model PRIVATE OpenMP::OpenMP_Fortran)
    endif()
endif()

# ------------------------------------------------------------
# Link options / libraries
# ------------------------------------------------------------
# NetCDF Fortran & C libs from nf-config / nc-config
# NF_FLIBS typically contains things like: -L<dir> -lnetcdff -lnetcdf ...
link_directories(${NC_LIBDIR})
target_link_libraries(model PRIVATE ${NF_FLIBS} MPI::MPI_Fortran)

# ------------------------------------------------------------
# Generate documentation
# ------------------------------------------------------------
find_package(Doxygen)
if(Doxygen_FOUND)
    set(DOXYGEN_IN  ${CMAKE_SOURCE_DIR}/doc/Doxyfile) # original config file
    set(DOXYGEN_OUT ${CMAKE_BINARY_DIR}/doc/Doxyfile) # where to store inside 'build' fol
    set(DOXYGEN_STYLE1_IN  ${CMAKE_SOURCE_DIR}/doc/custom.css) # original config file
    set(DOXYGEN_STYLE1_OUT ${CMAKE_BINARY_DIR}/doc/custom.css) # where to store inside 'b
    set(DOXYGEN_STYLE2_IN  ${CMAKE_SOURCE_DIR}/doc/custom_dark_theme.css) # original conf
    set(DOXYGEN_STYLE2_OUT ${CMAKE_BINARY_DIR}/doc/custom_dark_theme.css) # where to stor

    configure_file(${DOXYGEN_IN} ${DOXYGEN_OUT} @ONLY)
    configure_file(${DOXYGEN_STYLE1_IN} ${DOXYGEN_STYLE1_OUT} @ONLY)
    configure_file(${DOXYGEN_STYLE2_IN} ${DOXYGEN_STYLE2_OUT} @ONLY)

    add_custom_target(doc
        COMMAND ${DOXYGEN_EXECUTABLE} ${DOXYGEN_OUT}
        WORKING_DIRECTORY ${CMAKE_BINARY_DIR}/doc # where to save the generated docs
        COMMENT "Generating API documentation with Doxygen"
        VERBATIM
    )
endif()

# ------------------------------------------------------------
# Test setup
# ------------------------------------------------------------
enable_testing()

# If reference NetCDF files are in the source dir, copy them to the build dir
# so the test can find them.
file(GLOB REF_NC_FILES
    "${CMAKE_CURRENT_SOURCE_DIR}/output-serial.nc"
    "${CMAKE_CURRENT_SOURCE_DIR}/output-serial-optimized.nc"
)
if(REF_NC_FILES)
    file(COPY ${REF_NC_FILES} DESTINATION ${CMAKE_CURRENT_BINARY_DIR})
endif()

add_test(
    NAME model_test
    COMMAND bash -c
    "echo \"TEST: Running model...\"; \
     OUTPUT=\$(mpirun -np 4 ./model 100 1000 10); \
     MASS=\$(echo \"\${OUTPUT}\" | awk '/Fractional Delta Mass/{print \$NF}'); \
```

Profiling of sequential model

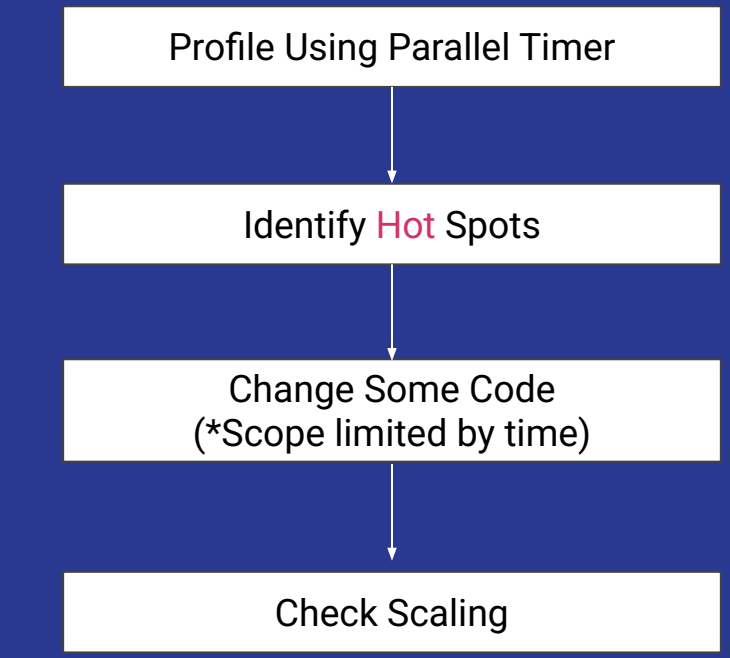
The script runs in 256 seconds, performing 2 trillion instructions where the processor is working 100% of the time.



```
Performance counter stats for './model':

    256,493.29 msec task-clock                #    0.999 CPUs utilized
         1,404      context-switches          #    5.474 /sec
             0      cpu-migrations            #    0.000 /sec
         7,531      page-faults               #   29.361 /sec
   841,132,451,095      cycles                #    3.279 GHz                      (38.46%)
 2,019,228,482,503      instructions          #    2.40  insn per cycle           (46.15%)
   154,293,868,322      branches              #  601.551 M/sec                    (53.85%)
    83,498,205      branch-misses             #    0.05% of all branches          (61.54%)
 4,206,179,864,501      slots                 #   16.399 G/sec                    (69.23%)
   488,849,685,816      topdown-retiring      #     10.8% retiring                (69.23%)
 3,381,438,714,598      topdown-bad-spec      #     74.6% bad speculation         (69.23%)
    33,118,518,795      topdown-fe-bound      #      0.7% frontend bound          (69.23%)
   627,311,712,968      topdown-be-bound      #     13.8% backend bound           (69.23%)
   587,568,847,051      L1-dcache-loads       #    2.291 G/sec                    (69.23%)
    17,681,480,301      L1-dcache-load-misses #    3.01% of all L1-dcache accesses  (69.23%)
       139,856,681      LLC-loads             #  545.264 K/sec                    (69.23%)
         4,278,813      LLC-load-misses       #    3.06% of all LL-cache accesses  (69.23%)
   <not supported>      L1-icache-loads
        59,686,921      L1-icache-load-misses                                   (30.77%)
   588,519,439,928      dTLB-loads            #    2.294 G/sec                    (30.77%)
         6,038,449      dTLB-load-misses      #    0.00% of all dTLB cache accesses  (30.77%)
   <not supported>      iTLB-loads
            47,368      iTLB-load-misses                                        (30.77%)
   <not supported>      L1-dcache-prefetches
   <not supported>      L1-dcache-prefetch-misses

     256.632430113 seconds time elapsed

     254.315529000 seconds user
       0.208176000 seconds sys
```

Profiling Results

Benchmark Workflow



How to time?

```
! --- The Scoped Timer Object ---
type :: CTimer
    integer :: db_index = 0
    real(8) :: start_time = 0.0d0
contains
    procedure :: start => start_timer_sub

    procedure :: stop => stop_timer

    final :: stop_timer_final
end type CTimer

interface CTimer
    module procedure new_timer
end interface
```

Source Code

What to time?

Computation &
Communication

*Not File I/O,
Not in this Step.

COMMUNICATION

Source Code

```
call mpi_timer%start("MPI: Communication")

  ! Non-blocking send/receive with neighbors
  call MPI_Irecv(recv_left, send_size, MPI_DOUBLE_PRECISION, &
                 left_rank, 1, MPI_COMM_WORLD, req(1), ierr)
  call MPI_Irecv(recv_right, send_size, MPI_DOUBLE_PRECISION, &
                 right_rank, 2, MPI_COMM_WORLD, req(2), ierr)
  call MPI_Isend(send_right, send_size, MPI_DOUBLE_PRECISION, &
                 right_rank, 1, MPI_COMM_WORLD, req(3), ierr)
  call MPI_Isend(send_left, send_size, MPI_DOUBLE_PRECISION, &
                 left_rank, 2, MPI_COMM_WORLD, req(4), ierr)

  call MPI_Waitall(4, req, status, ierr)

call mpi_timer%stop()
```

```
call mpi_timer%start("MPI: Communication")
! Reduce to global values across all MPI processes
call MPI_Allreduce(local_mass, mass, 1, MPI_DOUBLE_PRECISION, MPI_SUM, MPI_COMM_WORLD, ierr)
call MPI_Allreduce(local_te, te, 1, MPI_DOUBLE_PRECISION, MPI_SUM, MPI_COMM_WORLD, ierr)
call mpi_timer%stop()
```

```
subroutine rungekutta(s0,s1,fl,tend,dt)
  implicit none

  type(CTimer) :: pll_timer
  type(atmospheric_state), intent(inout) :: s0
  type(atmospheric_state), intent(inout) :: s1
  type(atmospheric_flux), intent(inout) :: fl
  type(atmospheric_tendency), intent(inout) :: tend
  real(wp), intent(in) :: dt
  real(wp) :: dt1, dt2, dt3
  logical, save :: dimswitch = .true.

  call pll_timer%start("Computation: rungekutta")

  dt1 = dt/1.0_wp
  dt2 = dt/2.0_wp
  dt3 = dt/3.0_wp
  if ( dimswitch ) then
    call step(s0, s0, s1, dt3, DIR_X, fl, tend)
    call step(s0, s1, s1, dt2, DIR_X, fl, tend)
    call step(s0, s1, s0, dt1, DIR_X, fl, tend)
    call step(s0, s0, s1, dt3, DIR_Z, fl, tend)
    call step(s0, s1, s1, dt2, DIR_Z, fl, tend)
    call step(s0, s1, s0, dt1, DIR_Z, fl, tend)
  else
    call step(s0, s0, s1, dt3, DIR_Z, fl, tend)
    call step(s0, s1, s1, dt2, DIR_Z, fl, tend)
    call step(s0, s1, s0, dt1, DIR_Z, fl, tend)
    call step(s0, s0, s1, dt3, DIR_X, fl, tend)
    call step(s0, s1, s1, dt2, DIR_X, fl, tend)
    call step(s0, s1, s0, dt1, DIR_X, fl, tend)
  end if
  dimswitch = .not. dimswitch
end subroutine rungekutta
```

COMPUTATION

```
subroutine thermal(x,z,r,u,w,t,hr,ht)
  implicit none
  type(CTimer) :: pll_timer
  real(wp), intent(in) :: x, z
  real(wp), intent(out) :: r, u, w, t
  real(wp), intent(out) :: hr, ht
  call pll_timer%start("Computation: thermal")
  call hydrostatic_const_theta(z,hr,ht)
  r = 0.0_wp
  t = 0.0_wp
  u = 0.0_wp
  w = 0.0_wp
  t = t + ellipse(x,z,3.0_wp,hxlen,p1,p1,p1)
end subroutine thermal
```


```
subroutine thermal(x,z,r,u,w,t,hr,ht)
  implicit none
  type(CTimer) :: pll_timer
  real(wp), intent(in) :: x, z
  real(wp), intent(out) :: r, u, w, t
  real(wp), intent(out) :: hr, ht
  call pll_timer%start("Computation: thermal")
  call hydrostatic_const_theta(z,hr,ht)
  r = 0.0_wp
  t = 0.0_wp
  u = 0.0_wp
  w = 0.0_wp
  t = t + ellipse(x,z,3.0_wp,hxlen,p1,p1,p1)
end subroutine thermal
```

Where is it slow?

```
-------------------------------------------------------------------------------------------------
PARALLEL TIMING STATISTICS (Microseconds)
                        Function      Max Total       Max Excl      Avg Total     Calls
-------------------------------------------------------------------------------------------------
                            INIT      170517.02       29747.99      170517.02         1
              Computation: thermal      140769.03       61001.37      140769.03    742557
Computation: hydrostatic_const       79767.66       79767.66       79767.66    742557
Computation: total_mass_energy        6187.96        6132.18        6187.96         2
          MPI: Communication             55.78          55.78          55.78         2
       Computation: rungekutta   190094693.01       15263.01   190094693.01      6001
            Computation: step   190079430.00   190079430.00   190079430.00     36006
-------------------------------------------------------------------------------------------------
 * Max Total/Excl: The slowest rank for that function.
 * Avg Total: Average wall time across all ranks.
USED CPU TIME:             190.101820 seconds
```

** Nx = 400, Nodes = 1, MPI Ranks = 1, Threads = 1

```
  call atmostat%exchange_halo_x( )

  hv_coef = -hv_beta * dx / (16.0_wp*dt)
  do k = 1, nz
    do i = 1, nx+1
      do ll = 1, NVARS
        do s = 1, STEN_SIZE
          stencil(s) = atmostat%mem(i-hs-1+s,k,ll)
        end do
        vals(ll) = - 1.0_wp * stencil(1)/12.0_wp &
                   + 7.0_wp * stencil(2)/12.0_wp &
                   + 7.0_wp * stencil(3)/12.0_wp &
                   - 1.0_wp * stencil(4)/12.0_wp
        d3_vals(ll) = - 1.0_wp * stencil(1) &
                      + 3.0_wp * stencil(2) &
                      - 3.0_wp * stencil(3) &
                      + 1.0_wp * stencil(4)
      end do
      r = vals(I_DENS) + ref%density(k)
      u = vals(I_UMOM) / r
      w = vals(I_WMOM) / r
      t = ( vals(I_RHOT) + ref%denstheta(k) ) / r
      p = c0*(r*t)**cdocv
      flux%dens(i,k) = r*u - hv_coef*d3_vals(I_DENS)
      flux%umom(i,k) = r*u*u+p - hv_coef*d3_vals(I_UMOM)
      flux%wmom(i,k) = r*u*w - hv_coef*d3_vals(I_WMOM)
      flux%rhot(i,k) = r*u*t - hv_coef*d3_vals(I_RHOT)
    end do
  end do
  do ll = 1, NVARS
    do k = 1, nz
      do i = 1, nx
        tendency%mem(i,k,ll) = &
            -( flux%mem(i+1,k,ll) - flux%mem(i,k,ll) ) / dx
      end do
    end do
  end do
end subroutine xtend
```

Turn serial execution to parallel (ILP)

```
  !$omp parallel do default(shared) &
  !$omp private(i, k, ll, s, stencil, vals, d3_vals, r, u, w, t, p)
  do k = 1, nz
    do i = 1, nx+1
      do ll = 1, NVARS
        do s = 1, STEN_SIZE
          stencil(s) = atmostat%mem(i-hs-1+s,k,ll)
        end do
        vals(ll) = - 1.0_wp * stencil(1)/12.0_wp &
                   + 7.0_wp * stencil(2)/12.0_wp &
                   + 7.0_wp * stencil(3)/12.0_wp &
                   - 1.0_wp * stencil(4)/12.0_wp
        d3_vals(ll) = - 1.0_wp * stencil(1) &
                      + 3.0_wp * stencil(2) &
                      - 3.0_wp * stencil(3) &
                      + 1.0_wp * stencil(4)
      end do
      r = vals(I_DENS) + ref%density(k)
      u = vals(I_UMOM) / r
      w = vals(I_WMOM) / r
      t = ( vals(I_RHOT) + ref%denstheta(k) ) / r
      p = c0*(r*t)**cdocv
      flux%dens(i,k) = r*u - hv_coef*d3_vals(I_DENS)
      flux%umom(i,k) = r*u*u+p - hv_coef*d3_vals(I_UMOM)
      flux%wmom(i,k) = r*u*w - hv_coef*d3_vals(I_WMOM)
      flux%rhot(i,k) = r*u*t - hv_coef*d3_vals(I_RHOT)
    end do
  end do
  !$omp end parallel do

  !$omp parallel do default(shared) private(i, k, ll) collapse(2)
  do ll = 1, NVARS
    do k = 1, nz
      do i = 1, nx
        tendency%mem(i,k,ll) = &
            -( flux%mem(i+1,k,ll) - flux%mem(i,k,ll) ) / dx
      end do
    end do
  end do
  !$omp end parallel do
```

```
  call atmostat%exchange_halo_z(ref)

  hv_coef = -hv_beta * dz / (16.0_wp*dt)
  do k = 1, nz+1
    do i = 1, nx
      do ll = 1, NVARS
        do s = 1, STEN_SIZE
          stencil(s) = atmostat%mem(i,k-hs-1+s,ll)
        end do
        vals(ll) = - 1.0_wp * stencil(1)/12.0_wp &
                   + 7.0_wp * stencil(2)/12.0_wp &
                   + 7.0_wp * stencil(3)/12.0_wp &
                   - 1.0_wp * stencil(4)/12.0_wp
        d3_vals(ll) = - 1.0_wp * stencil(1) &
                      + 3.0_wp * stencil(2) &
                      - 3.0_wp * stencil(3) &
                      + 1.0_wp * stencil(4)
      end do
      r = vals(I_DENS) + ref%idens(k)
      u = vals(I_UMOM) / r
      w = vals(I_WMOM) / r
      t = ( vals(I_RHOT) + ref%idenstheta(k) ) / r
      p = c0*(r*t)**cdocv - ref%pressure(k)
      if (k == 1 .or. k == nz+1) then
        w = 0.0_wp
        d3_vals(I_DENS) = 0.0_wp
      end if
      flux%dens(i,k) = r*w - hv_coef*d3_vals(I_DENS)
      flux%umom(i,k) = r*w*u - hv_coef*d3_vals(I_UMOM)
      flux%wmom(i,k) = r*w*w+p - hv_coef*d3_vals(I_WMOM)
      flux%rhot(i,k) = r*w*t - hv_coef*d3_vals(I_RHOT)
    end do
  end do

  do ll = 1, NVARS
    do k = 1, nz
      do i = 1, nx
        tendency%mem(i,k,ll) = &
            -( flux%mem(i,k+1,ll) - flux%mem(i,k,ll) ) / dz
        if (ll == I_WMOM) then
          tendency%wmom(i,k) = tendency%wmom(i,k) - atmostat%dens(i,k)*grav
        end if
      end do
    end do
  end do
end subroutine ztend
```

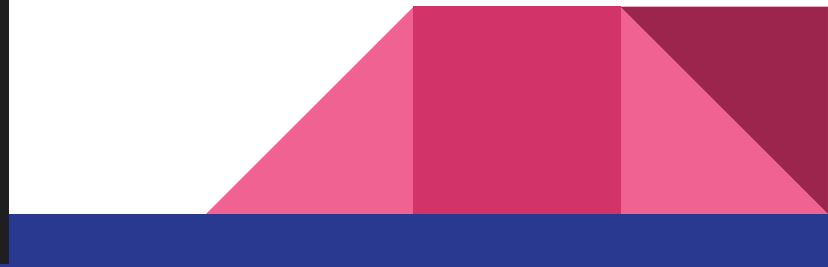
Turn serial execution to parallel (ILP)

```
  !$omp parallel do default(shared) &
  !$omp private(i, k, ll, s, stencil, vals, d3_vals, r, u, w, t, p)
  do k = 1, nz+1
    do i = 1, nx
      do ll = 1, NVARS
        do s = 1, STEN_SIZE
          stencil(s) = atmostat%mem(i,k-hs-1+s,ll)
        end do
        vals(ll) = - 1.0_wp * stencil(1)/12.0_wp &
                   + 7.0_wp * stencil(2)/12.0_wp &
                   + 7.0_wp * stencil(3)/12.0_wp &
                   - 1.0_wp * stencil(4)/12.0_wp
        d3_vals(ll) = - 1.0_wp * stencil(1) &
                      + 3.0_wp * stencil(2) &
                      - 3.0_wp * stencil(3) &
                      + 1.0_wp * stencil(4)
      end do
      r = vals(I_DENS) + ref%idens(k)
      u = vals(I_UMOM) / r
      w = vals(I_WMOM) / r
      t = ( vals(I_RHOT) + ref%idenstheta(k) ) / r
      p = c0*(r*t)**cdocv - ref%pressure(k)

      ! This IF statement is thread-safe because it relies only on 'k'
      if (k == 1 .or. k == nz+1) then
        w = 0.0_wp
        d3_vals(I_DENS) = 0.0_wp
      end if

      flux%dens(i,k) = r*w - hv_coef*d3_vals(I_DENS)
      flux%umom(i,k) = r*w*u - hv_coef*d3_vals(I_UMOM)
      flux%wmom(i,k) = r*w*w+p - hv_coef*d3_vals(I_WMOM)
      flux%rhot(i,k) = r*w*t - hv_coef*d3_vals(I_RHOT)
    end do
  end do
  !$omp end parallel do

  !$omp parallel do default(shared) private(i, k, ll) collapse(2)
  do ll = 1, NVARS
    do k = 1, nz
      do i = 1, nx
        tendency%mem(i,k,ll) = &
            -( flux%mem(i,k+1,ll) - flux%mem(i,k,ll) ) / dz

        ! Adding gravity source term
        if (ll == I_WMOM) then
          tendency%wmom(i,k) = tendency%wmom(i,k) - atmostat%dens(i,k)*grav
        end if
      end do
    end do
  end do
  !$omp end parallel do
#endif

end subroutine ztend
```

```
subroutine setup_domain_decomposition(rank, num_procs)
  implicit none
  integer, intent(in) :: rank, num_procs
  integer :: base_nx, remainder

  myrank = rank
  nprocs = num_procs
  nx_global = nx

  ! Divide nx among processes
  base_nx = nx_global / nprocs
  remainder = mod(nx_global, nprocs)

  ! Distribute remainder among first processes
  if (myrank < remainder) then
    nx_local = base_nx + 1
    i_start_global = myrank * (base_nx + 1) + 1
  else
    nx_local = base_nx
    i_start_global = remainder * (base_nx + 1) + (myrank - remainder) * base_nx + 1
  end if
  i_end_global = i_start_global + nx_local - 1

  ! Update nx to local value for this process
  nx = nx_local

  ! Neighbors (periodic in x)
  left_rank = mod(myrank - 1 + nprocs, nprocs)
  right_rank = mod(myrank + 1, nprocs)

end subroutine setup_domain_decomposition

end module dimensions
```

Divide Data to be processed among diff processing elements

(Data/Domain Decomposition)



```
! Non-blocking send/receive with neighbors
call MPI_Irecv(recv_left, send_size, MPI_DOUBLE_PRECISION, &
               left_rank, 1, MPI_COMM_WORLD, req(1), ierr)
call MPI_Irecv(recv_right, send_size, MPI_DOUBLE_PRECISION, &
               right_rank, 2, MPI_COMM_WORLD, req(2), ierr)
call MPI_Isend(send_right, send_size, MPI_DOUBLE_PRECISION, &
               right_rank, 1, MPI_COMM_WORLD, req(3), ierr)
call MPI_Isend(send_left, send_size, MPI_DOUBLE_PRECISION, &
               left_rank, 2, MPI_COMM_WORLD, req(4), ierr)
```

Use Non-blocking communication to exchange boundary conditions

*But you have to do Waitall

```
call mpi_timer%start("MPI: Communication")
! Reduce to global values across all MPI processes
call MPI_Allreduce(local_mass, mass, 1, MPI_DOUBLE_PRECISION, MPI_SUM, MPI_COMM_WORLD, ierr)
call MPI_Allreduce(local_te, te, 1, MPI_DOUBLE_PRECISION, MPI_SUM, MPI_COMM_WORLD, ierr)
call mpi_timer%stop()

end subroutine total_mass_energy
```

Sum mass and energy from all processes

Resource: Leonardo, Booster Partition Specs:

Booster | DCGP

Type	Specific
Models	Atos BullSequana X2135, Da Vinci single-node GPU
Racks	116
Nodes	3456
Processors/node	1x Intel Ice Lake Intel Xeon Platinum 8358
CPU/node	32
Accelerators/node	4x NVIDIA Ampere100 custom, 64GiB HBM2e NVLink 3.0 (200 GB/s)
Local Storage/node (tmfs)	(none)
RAM/node	512 GiB DDR4 3200 MHz
Rmax	241.2 PFlop/s (top500)
Internal Network	200 Gbps NVIDIA Mellanox HDR InfiniBand - Dragonfly+ Topology
Storage (raw capacity)	106 PiB based on DDN ES7990X and Hard Drive Disks (Capacity Tier) 5.7 PiB based on DDN ES400NVX2 and Solid State Drives (Fast Tier)

Slurm Config

```
# 1. Load Modules
module purge
module load gcc/12.2.0
module load openmpi/4.1.6--gcc--12.2.0-cuda-12.2
module load netcdf-fortran/4.6.1--openmpi--4.1.6--gcc--12.2.0-spack0.22
```

```
# 2. Setup Environment Variables
export OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK}
export OMP_PROC_BIND=close
export OMP_PLACES=cores
```

```
# 3. Execution

# Grid Size = 400
# Total Time Step = 1000
srun ./model 400 1000
```

```
#!/bin/bash
#SBATCH -A ICT25_MHPC_0
#SBATCH --partition=boost_usr_prod
##SBATCH --qos=boost_qos_dbg
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=2
#SBATCH --cpus-per-task=56
#SBATCH --time=00:10:00
#SBATCH --job-name=atmos_omp
#SBATCH --hint=nomultithread
#SBATCH --output=logs/%x_%j.out
#SBATCH --error=logs/%x_%j.err
```

Source Code

Baseline

```
--- Running Simulation ---
Nodes = 1
Tasks / Node = 1 (Pure OpenMP)
CPUs / Task = 1 (OMP Threads)
 ================= Execution Info ==================
  Number of MPI tasks:    1
  Number of OpenMP threads:    1
   OpenMP: ENABLED
   OpenACC: DISABLED
 ===================================================
 --------------- Domain Decomposition ---------------
  Global nx:    400
  Local nx per process: ~   400
 ----------------------------------------------------


 ----------------- Atmosphere check -----------------
 Fractional Delta Mass  :    1.9541502512719520E-016
 Fractional Delta Energy:    1.1896204747724564E-004
 ----------------------------------------------------
 ------------------------------------------------------------------------------------------------------------
 PARALLEL TIMING STATISTICS (Microseconds)
                    Function      Max Total       Max Excl      Avg Total     Calls
 ------------------------------------------------------------------------------------------------------------
                        INIT      170517.02       29747.99      170517.02         1
           Computation: thermal      140769.03       61001.37      140769.03    742557
Computation: hydrostatic_const       79767.66       79767.66       79767.66    742557
Computation: total_mass_energy        6187.96        6132.18        6187.96         2
            MPI: Communication          55.78          55.78          55.78         2
      Computation: rungekutta   190094693.01       15263.01   190094693.01      6001
           Computation: step   190079430.00   190079430.00   190079430.00     36006
 ------------------------------------------------------------------------------------------------------------
 * Max Total/Excl: The slowest rank for that function.
 * Avg Total: Average wall time across all ranks.
 SIMPLE ATMOSPHERIC MODEL RUN COMPLETED.
Wall Clock End:   14:33:28.704
USED CPU TIME:            190.101820 seconds
--- Ending Simulation ---
```

Best Case

*On 1 Node, 1 Process,

32 Threads
(openmp only)

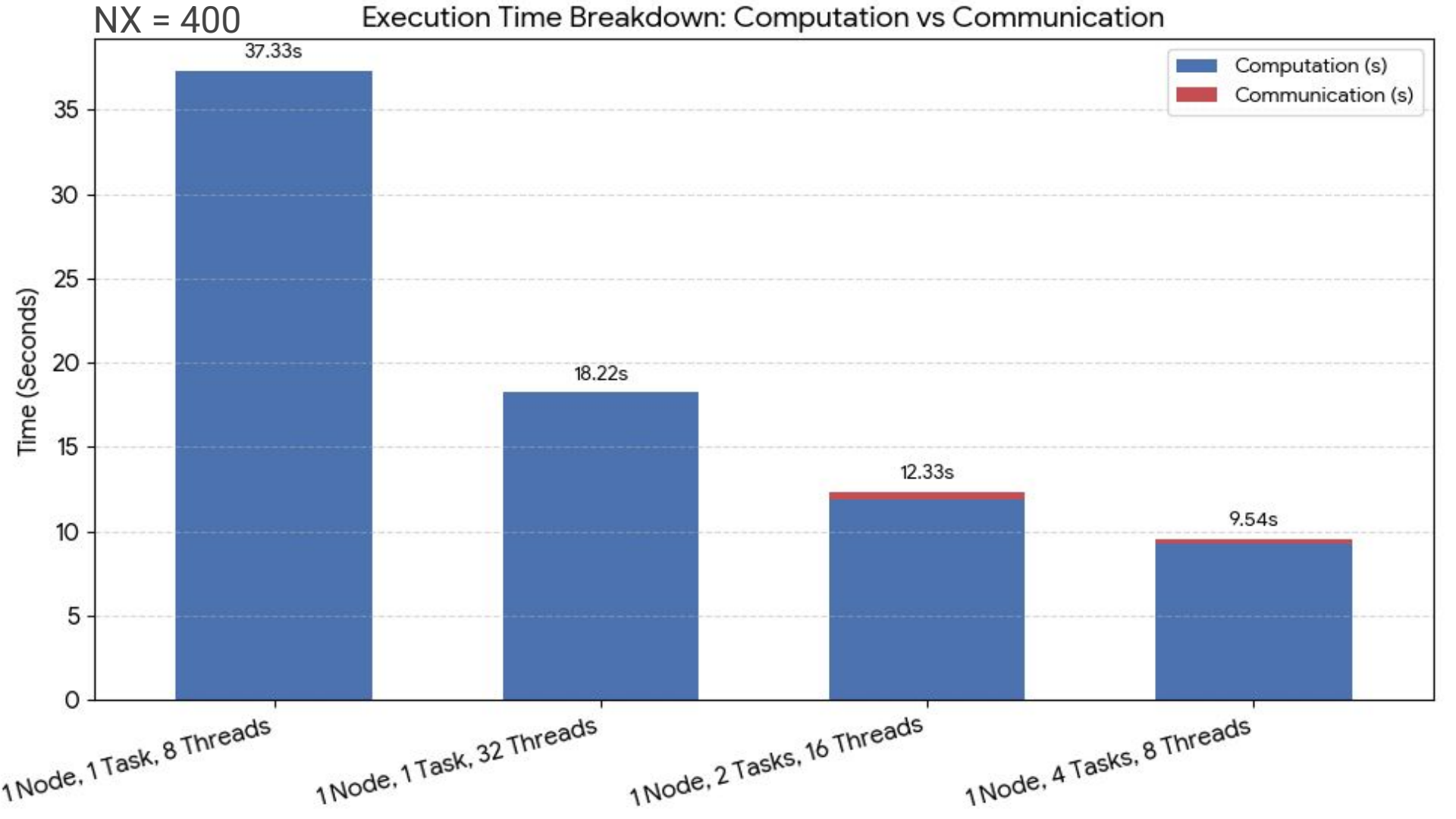
```
--- Running Simulation ---
Nodes = 1
Tasks / Node = 1 (Pure OpenMP)
CPUs / Task = 32 (OMP Threads)
 ================= Execution Info ==================
  Number of MPI tasks:    1
  Number of OpenMP threads:   32
   OpenMP: ENABLED
   OpenACC: DISABLED
 ===================================================
 --------------- Domain Decomposition ---------------
  Global nx:    400
  Local nx per process: ~   400
 ---------------------------------------------------
 SIMPLE ATMOSPHERIC MODEL STARTING.
Wall Clock Start: 14:47:18.499
 ----------------- Atmosphere check -----------------
 Fractional Delta Mass  :    1.9541502512719520E-016
 Fractional Delta Energy:    1.1896204747724564E-004
 ---------------------------------------------------
 ------------------------------------------------------------------------------------------------------
 PARALLEL TIMING STATISTICS (Microseconds)
                    Function      Max Total       Max Excl      Avg Total     Calls
 ------------------------------------------------------------------------------------------------------
                        INIT      165067.33       28435.45      165067.33         1
          Computation: thermal      136631.88       59025.72      136631.88    742557
Computation: hydrostatic_const       77606.15       77606.15       77606.15    742557
Computation: total_mass_energy        6406.42        6290.12        6406.42         2
           MPI: Communication         116.31         116.31         116.31         2
       Computation: rungekutta    18077086.59        6401.51    18077086.59      6001
            Computation: step    18070685.08    18070685.08    18070685.08     36006
 ------------------------------------------------------------------------------------------------------
 * Max Total/Excl: The slowest rank for that function.
 * Avg Total: Average wall time across all ranks.
 SIMPLE ATMOSPHERIC MODEL RUN COMPLETED.
Wall Clock End:   14:47:36.750
USED CPU TIME:             18.082526 seconds
--- Ending Simulation ---
```

Best Case

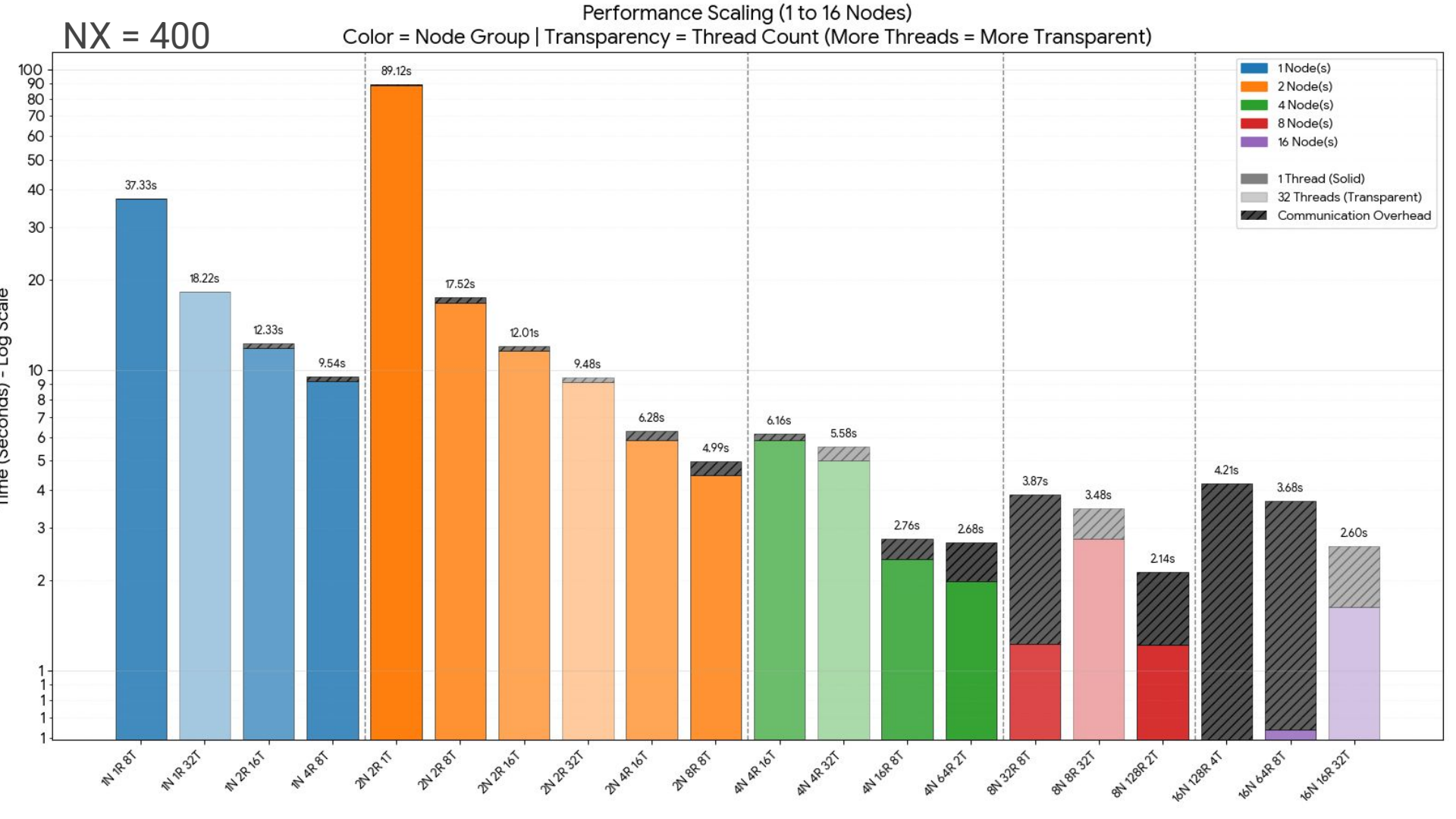
*On 1 Node, MPI + openmp

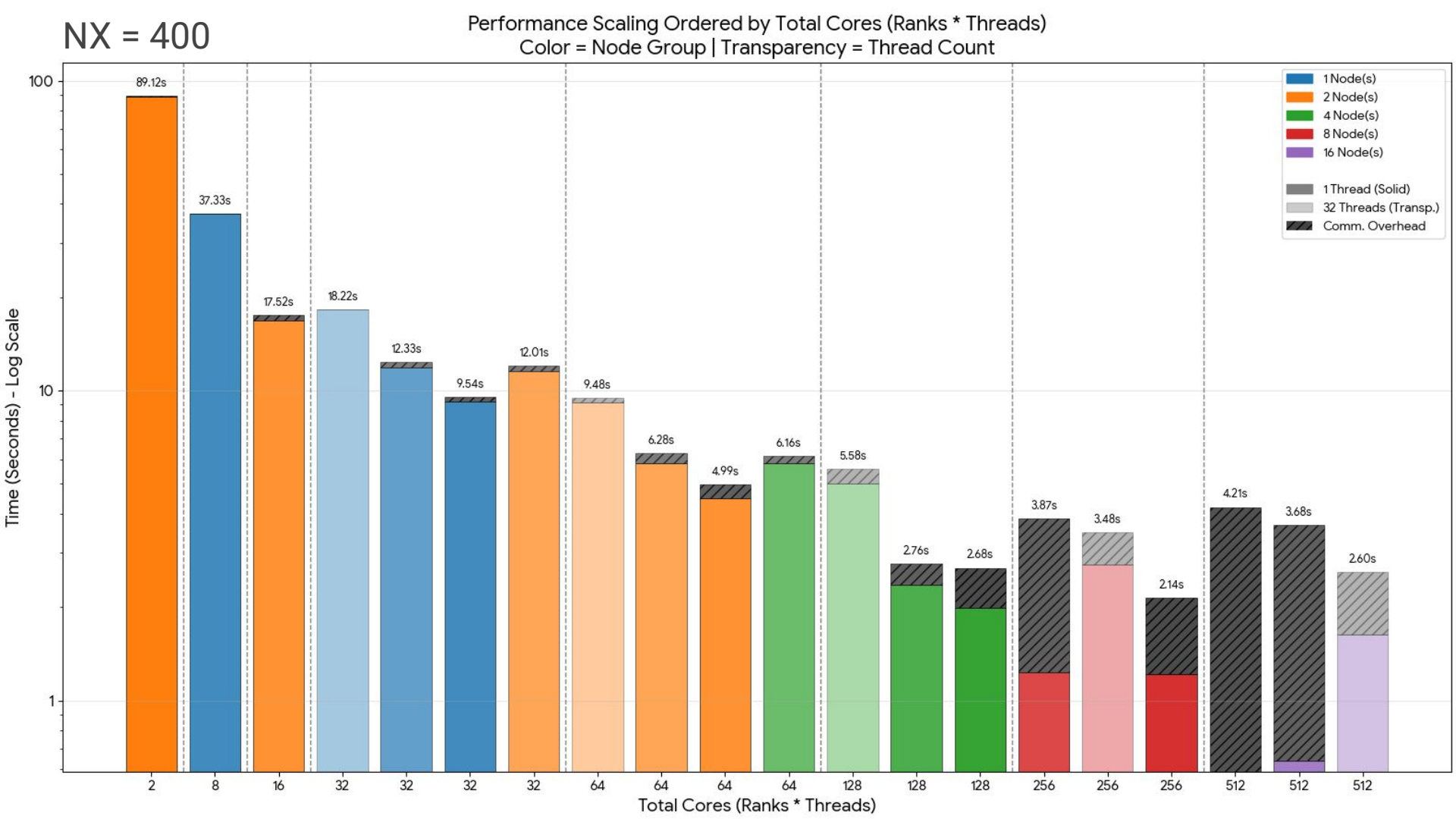
```
--- Running Simulation ---
Nodes = 1
Tasks / Node = 1 (Pure OpenMP)
CPUs / Task = 8 (OMP Threads)
 ================ Execution Info ================
  Number of MPI tasks:    4
  Number of OpenMP threads:    8
   OpenMP: ENABLED
   OpenACC: DISABLED
 ================================================
 --------------- Domain Decomposition --------------
  Global nx:    400
  Local nx per process: ~   100
 -------------------------------------------------
 SIMPLE ATMOSPHERIC MODEL STARTING.
Wall Clock Start: 10:09:30.864
 ----------------- Atmosphere check ----------------
 Fractional Delta Mass  :   -3.9083005025439021E-016
 Fractional Delta Energy:    1.1896204747669450E-004
 -------------------------------------------------
 ------------------------------------------------------------------------------------------------------------
 PARALLEL TIMING STATISTICS (Microseconds)
                                  Function      Max Total       Max Excl      Avg Total     Calls
 ------------------------------------------------------------------------------------------------------------
                                      INIT       43249.76        7368.82       42773.03         4
                        Computation: thermal       35927.44       15835.81       35442.39    767028
Computation: hydrostatic_const             20118.05       20118.05       20100.00    767028
Computation: total_mass_energy              4093.98        1578.41        3712.81         8
                     MPI: Communication     312385.16      312385.16      247794.59     72020
                Computation: rungekutta    9341487.32        5612.23     9341320.38     24004
                     Computation: step     9336195.37     9183962.42     9335945.59    144024
 ------------------------------------------------------------------------------------------------------------
 * Max Total/Excl: The slowest rank for that function.
 * Avg Total: Average wall time across all ranks.
 SIMPLE ATMOSPHERIC MODEL RUN COMPLETED.
Wall Clock End:   10:09:40.253
USED CPU TIME:              9.343526 seconds
```

NX = 400


Execution Time Breakdown: Computation vs Communication
Computation (s)
Communication (s)
Time (Seconds)
0
5
10
15
20
25
30
35
37.33s
18.22s
12.33s
9.54s
1 Node, 1 Task, 8 Threads
1 Node, 1 Task, 32 Threads
1 Node, 2 Tasks, 16 Threads
1 Node, 4 Tasks, 8 Threads

If the budget allows, use multiple nodes


NX = 400
Performance Scaling (1 to 16 Nodes)
Color = Node Group | Transparency = Thread Count (More Threads = More Transparent)
Time (Seconds) - Log Scale
1 Node(s)
2 Node(s)
4 Node(s)
8 Node(s)
16 Node(s)
1 Thread (Solid)
32 Threads (Transparent)
Communication Overhead
37.33s
18.22s
12.33s
9.54s
89.12s
17.52s
12.01s
9.48s
6.28s
4.99s
6.16s
5.58s
2.76s
2.68s
3.87s
3.48s
2.14s
4.21s
3.68s
2.60s
1N 1R 8T
1N 1R 32T
1N 2R 16T
1N 4R 8T
2N 2R 1T
2N 2R 8T
2N 2R 16T
2N 2R 32T
2N 4R 16T
2N 8R 8T
4N 4R 16T
4N 4R 32T
4N 16R 8T
4N 64R 2T
8N 32R 8T
8N 8R 32T
8N 128R 2T
16N 128R 4T
16N 64R 8T
16N 16R 32T


NX = 400
Performance Scaling Ordered by Total Cores (Ranks * Threads)
Color = Node Group | Transparency = Thread Count
1 Node(s)
2 Node(s)
4 Node(s)
8 Node(s)
16 Node(s)
1 Thread (Solid)
32 Threads (Transp.)
Comm. Overhead
Time (Seconds) - Log Scale
100
10
1
89.12s
37.33s
17.52s
18.22s
12.33s
9.54s
12.01s
9.48s
6.28s
4.99s
6.16s
5.58s
2.76s
2.68s
3.87s
3.48s
2.14s
4.21s
3.68s
2.60s
2
8
16
32
32
32
32
64
64
64
64
128
128
128
256
256
256
512
512
512
Total Cores (Ranks * Threads)

OpenACC

How is the data move to the device?

```
#ifdef USE_OPENACC
  call nvtx_push("Data Transfer to GPU")
  !$acc enter data create(oldstat, newstat, flux, tend, ref)
  !$acc enter data copyin(oldstat%mem, newstat%mem, flux%mem, tend%mem)
  !$acc enter data copyin(ref%density, ref%denstheta, ref%idens, ref%idenstheta, ref%pressure)
  !$acc enter data attach(oldstat%mem, newstat%mem, flux%mem, tend%mem)
  !$acc enter data attach(ref%density, ref%denstheta, ref%idens, ref%idenstheta, ref%pressure)
  call nvtx_pop()
#endif
```

```
#ifdef USE_OPENACC
      call nvtx_push("Data Update from GPU")
      !$acc update self(oldstat%mem)
      call nvtx_pop()
#endif
        ! --- For benchmark, removing file i/o
      call write_record(oldstat,ref,etime)
    end if
```

```
#ifdef USE_OPENACC
  call nvtx_push("Data Cleanup from GPU")
  !$acc exit data delete(oldstat%mem, newstat%mem, flux%mem, tend%mem)
  !$acc exit data delete(ref%density, ref%denstheta, ref%idens, ref%idenstheta, ref%pressure)
  !$acc exit data delete(oldstat, newstat, flux, tend, ref)
  call nvtx_pop()
#endif
```

E

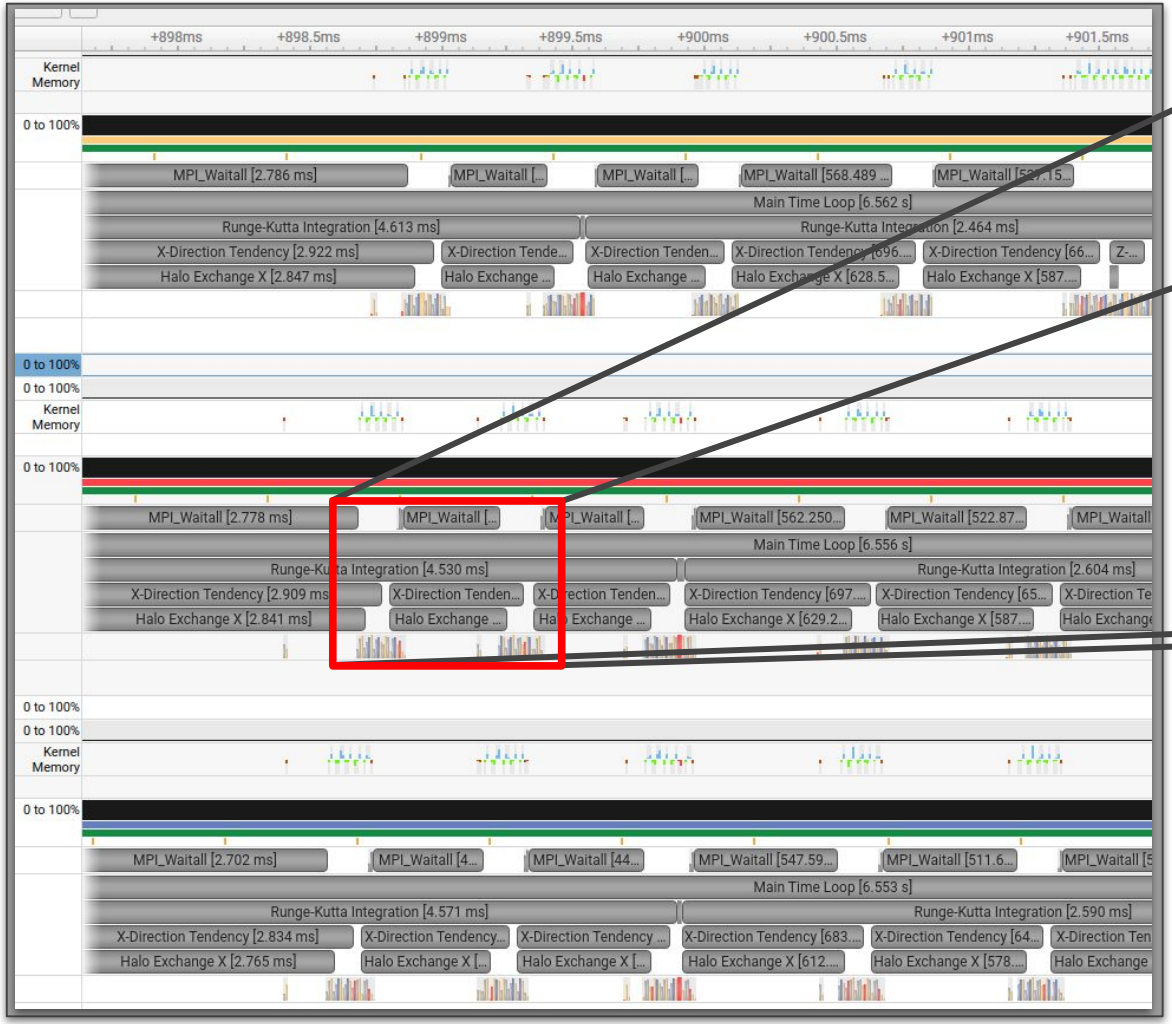
How was the code parallelized?

```
#ifdef USE_OPENACC
      !$acc parallel loop collapse(3) present(s0%mem, tend%mem, s2%mem)
      do ll = 1, NVARS
        do k = 1, nz
          do i = 1, nx
            s2%mem(i,k,ll) = s0%mem(i,k,ll) + dt * tend%mem(i,k,ll)
          end do
        end do
      end do
      !$acc end parallel loop
#else
      do ll = 1, NVARS
        do k = 1, nz
          do i = 1, nx
            s2%mem(i,k,ll) = s0%mem(i,k,ll) + dt * tend%mem(i,k,ll)
          end do
        end do
      end do
#endif
```

```
call mpi_timer%start("MPI: Communication")
!$acc host_data use_device(send_left, send_right, recv_left, recv_right)
call MPI_Irecv(recv_left, send_size, MPI_DOUBLE_PRECISION, &
               left_rank, 1, MPI_COMM_WORLD, req(1), ierr)
call MPI_Irecv(recv_right, send_size, MPI_DOUBLE_PRECISION, &
               right_rank, 2, MPI_COMM_WORLD, req(2), ierr)
call MPI_Isend(send_right, send_size, MPI_DOUBLE_PRECISION, &
               right_rank, 1, MPI_COMM_WORLD, req(3), ierr)
call MPI_Isend(send_left, send_size, MPI_DOUBLE_PRECISION, &
               left_rank, 2, MPI_COMM_WORLD, req(4), ierr)
call MPI_Waitall(4, req, status, ierr)
!$acc end host_data
call mpi_timer%stop()
```

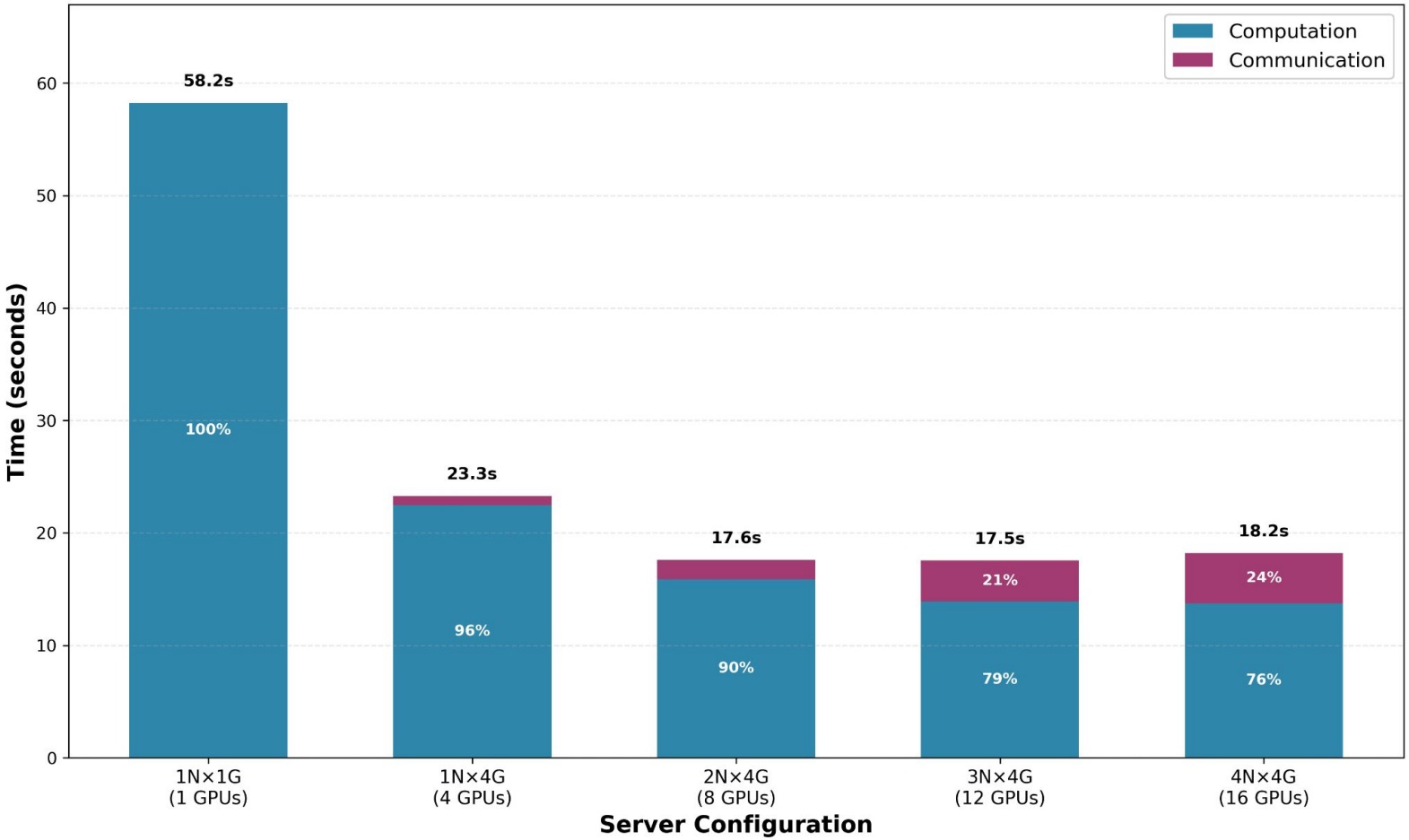
```
#ifdef USE_OPENACC
    !$acc parallel loop collapse(2) present(atmostat%mem, ref%density, ref%denstheta, flux%mem) &
    !$acc   private(stencil, vals, d3_vals, r, u, w, t, p)
    do k = 1, nz
      do i = 1, nx+1
        do ll = 1, NVARS
          do s = 1, STEN_SIZE
            stencil(s) = atmostat%mem(i-hs-1+s,k,ll)
          end do
          vals(ll) = - 1.0_wp * stencil(1)/12.0_wp &
                     + 7.0_wp * stencil(2)/12.0_wp &
                     + 7.0_wp * stencil(3)/12.0_wp &
                     - 1.0_wp * stencil(4)/12.0_wp
          d3_vals(ll) = - 1.0_wp * stencil(1) &
                        + 3.0_wp * stencil(2) &
                        - 3.0_wp * stencil(3) &
                        + 1.0_wp * stencil(4)
        end do
        r = vals(I_DENS) + ref%density(k)
        u = vals(I_UMOM) / r
        w = vals(I_WMOM) / r
        t = ( vals(I_RHOT) + ref%denstheta(k) ) / r
        p = c0*(r*t)**cdocv
        flux%mem(i,k,I_DENS) = r*u - hv_coef*d3_vals(I_DENS)
        flux%mem(i,k,I_UMOM) = r*u*u+p - hv_coef*d3_vals(I_UMOM)
        flux%mem(i,k,I_WMOM) = r*u*w - hv_coef*d3_vals(I_WMOM)
        flux%mem(i,k,I_RHOT) = r*u*t - hv_coef*d3_vals(I_RHOT)
      end do
    end do
    !$acc end parallel loop
```

E

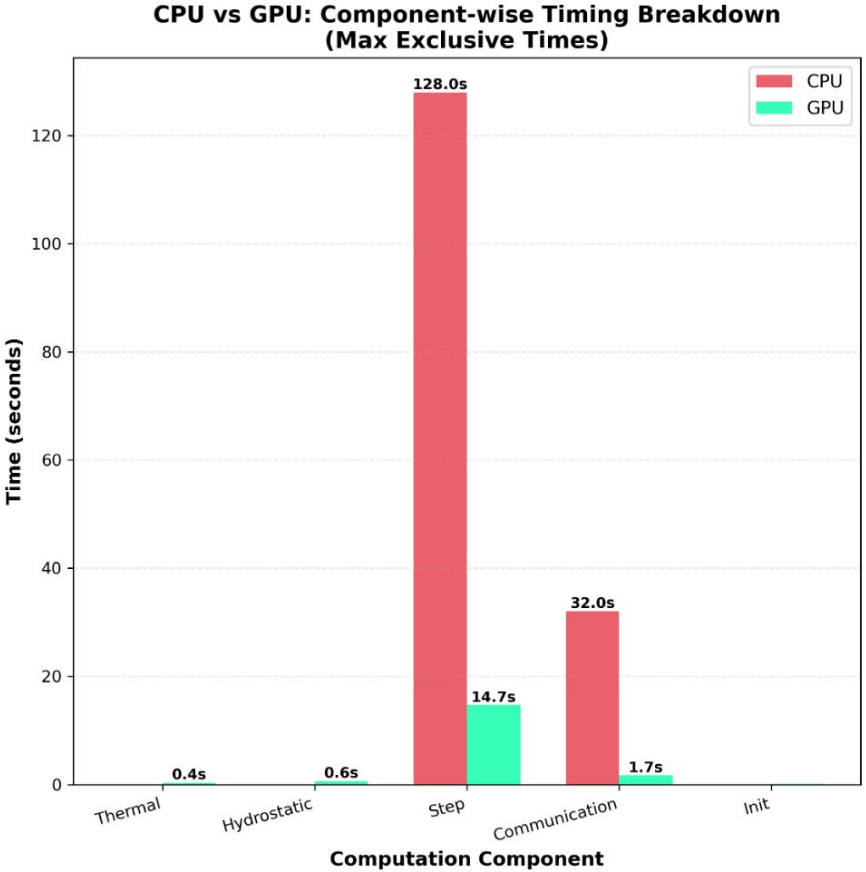


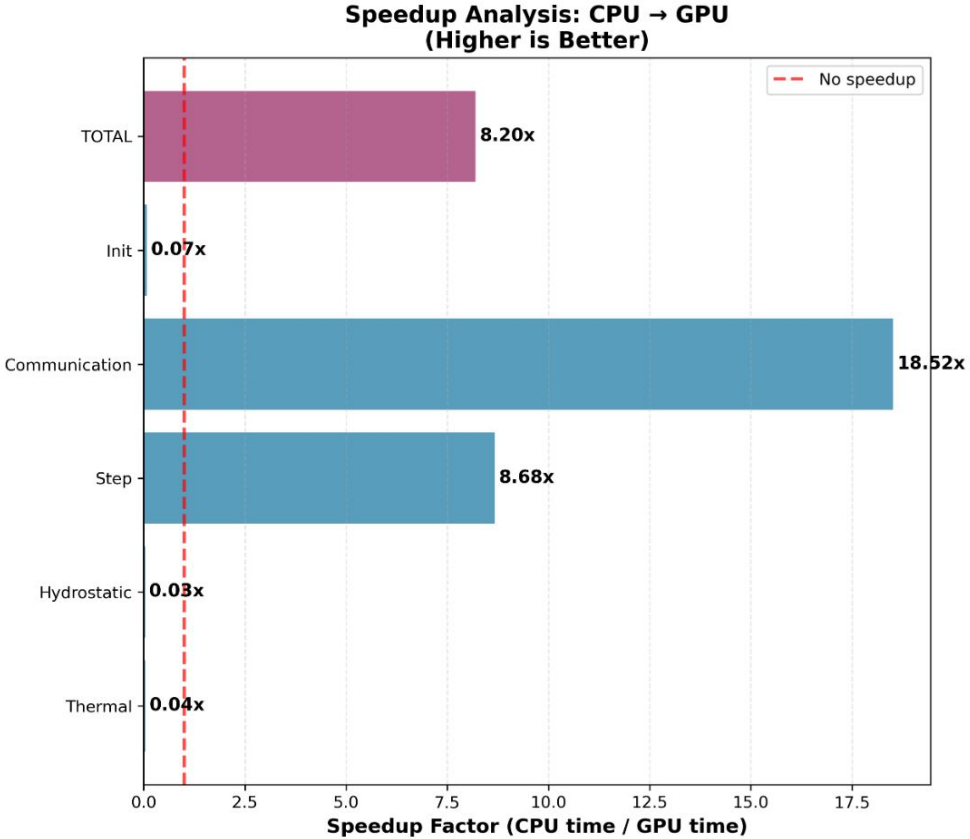
Profiling with Nsys

E


Multi-GPU Scaling: Computation vs Communication Time (nx=2000, nz=1000)
(Max Exclusive Times)
Computation
Communication
Time (seconds)
0
10
20
30
40
50
60
58.2s
100%
23.3s
96%
17.6s
90%
17.5s
21%
79%
18.2s
24%
76%
1N×1G
(1 GPUs)
1N×4G
(4 GPUs)
2N×4G
(8 GPUs)
3N×4G
(12 GPUs)
4N×4G
(16 GPUs)
Server Configuration

E





CPU: 128 MPI × 2 OMP = 256 cores | GPU: 8 MPI tasks × 2 nodes | Problem: nx=2000, nz=1000, t=1000.0s

E

Testing: energy budgets

Test will pass if `FDM < 1x10^-13` and `FDE < 1x10^-3`

```
 SIMPLE ATMOSPHERIC MODEL RUN COMPLETED.
Wall Clock End:   13:42:30.577
USED CPU TIME:                0.514330 seconds
Fractional Delta Mass   = 2.8139763618316794E-014
Fractional Delta Energy = 1.0051785903563540E-004
TEST PASSED.
```

Testing: output files

```
root@3f0e2852e6ec:/app# python3 nccmp3.py output-serial-400.nc output-serial-optimized-400.nc output-400-gpu1N-4R.nc
#################
Var Name            :  |1-2|                ,  |2-3|                ,  |2-3|/|1-2|
/app/nccmp3.py:88: RuntimeWarning: invalid value encountered in scalar divide
  ":  %20.10e  ,  %20.10e  ,  %20.10e"%(norm12,norm23,norm23/norm12))
rho                 :      0.0000000000e+00  ,      0.0000000000e+00  ,                  nan
u                   :      0.0000000000e+00  ,      0.0000000000e+00  ,                  nan
w                   :      0.0000000000e+00  ,      0.0000000000e+00  ,                  nan
theta               :      0.0000000000e+00  ,      0.0000000000e+00  ,                  nan
#################
root@3f0e2852e6ec:/app# python3 nccmp3.py output.nc output-serial-optimized-400.nc output-400-2N-8R-4T.nc
#################
Var Name            :  |1-2|                ,  |2-3|                ,  |2-3|/|1-2|
/app/nccmp3.py:88: RuntimeWarning: invalid value encountered in scalar divide
  ":  %20.10e  ,  %20.10e  ,  %20.10e"%(norm12,norm23,norm23/norm12))
rho                 :      0.0000000000e+00  ,      0.0000000000e+00  ,                  nan
u                   :      0.0000000000e+00  ,      0.0000000000e+00  ,                  nan
w                   :      0.0000000000e+00  ,      0.0000000000e+00  ,                  nan
theta               :      0.0000000000e+00  ,      0.0000000000e+00  ,                  nan
#################
```

2D dynamical fluid flow in Fortran

2-D compressible Euler solver with dimensional splitting, RK3 time stepping, and hyper-viscosity

Main Page | Modules | Data Types List | Files

- 2D dynamical fluid flow in Fortran
  - Modules
    - Modules List
      - calculation_types
      - dimensions
      - indexing
      - iodir
      - legendre_quadrature
      - module_output
      - module_physics
      - module_types
      - parallel_parameters
      - parallel_timer
      - physical_constants
      - physical_parameters
    - Module Members
  - Data Types List
  - Files

atmosphere_model()

```
program atmosphere_model
```

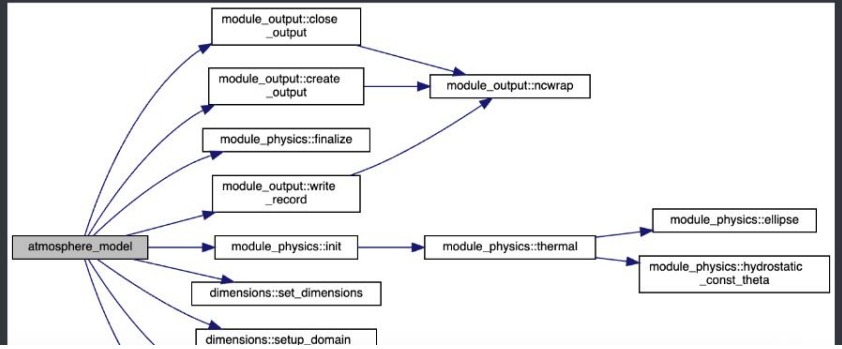
Main driver for the 2-D compressible Euler solver.

Initializes domain/physics, runs the Runge-Kutta time stepping with dimensional splitting, optionally outputs diagnostics, and reports mass/energy conservation plus wall-clock timing. Command line arguments allow overriding grid size, simulation length, and output frequency.

Definition at line **7** of file **model.F90**.

References **module_output::close_output()**, **module_output::create_output()**, **dimensions::default_nx**, **dimensions::default_output_freq**, **dimensions::default_sim_time**, **module_physics::dt**, **module_physics::finalize()**, **module_physics::flux**, **module_physics::init()**, **module_physics::newstat**, **dimensions::nprocs**, **dimensions::nx**, **dimensions::nx_global**, **module_physics::oldstat**, **dimensions::output_freq**, **module_physics::ref**, **module_physics::rungekutta()**, **dimensions::set_dimensions()**, **dimensions::setup_domain_decomposition()**, **dimensions::sim_time**, **iodir::stdout**, **module_physics::tend**, **module_physics::total_mass_energy()**, **calculation_types::wp**, and **module_output::write_record()**.

Here is the call graph for this function:



Added CMake target to generate docs using Doxygen

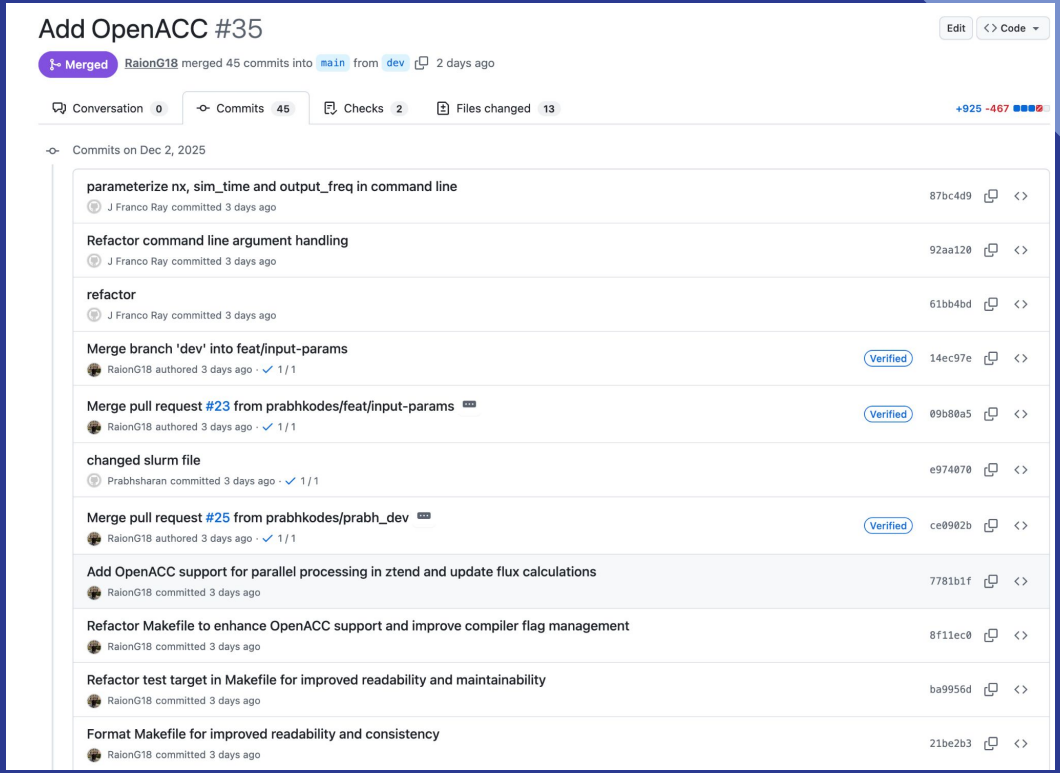
Retrospective

Make your life easier

1. Take input params from cli.
2. Use debug statements
3. Check your results every time.
4. Double check your benchmark configurations.
5. Prefer slurm managed configs.
6. Document your changes.
7. Have good commit info and linear history.
8. Communication is key.
9. Branch well, always pull the latest version.
10. Don’t ignore compilation warnings.
11. Code Readability >> Cleverness
12. Test locally and on cluster.
13. Profile your code even if it looks like it works.

```
# 3. Execution

# Grid Size = 400
# Total Time Step = 1000
srun ./model 400 1000
```

```
--- Running Simulation ---
Nodes = 1
Tasks / Node = 1 (Pure OpenMP)
CPUs / Task = 8 (OMP Threads)
 ================ Execution Info ================
  Number of MPI tasks:    1
  Number of OpenMP threads:    8
    OpenMP: ENABLED
    OpenACC: DISABLED
 ================================================
 -------------- Domain Decomposition --------------
  Global nx:    400
  Local nx per process: ~   400
 ------------------------------------------------
 SIMPLE ATMOSPHERIC MODEL STARTING.
Wall Clock Start: 14:46:06.408
 ---------------- Atmosphere check ----------------
 Fractional Delta Mass  :    1.9541502512719520E-016
 Fractional Delta Energy:    1.1896204747724564E-004
 ------------------------------------------------
 SIMPLE ATMOSPHERIC MODEL RUN COMPLETED.
Wall Clock End:   14:46:43.768
USED CPU TIME:              37.192299 seconds
--- Ending Simulation ---
```



Add OpenACC #35

Edit · Code

Merged · RaionG18 merged 45 commits into main from dev · 2 days ago

Conversation 0 · Commits 45 · Checks 2 · Files changed 13 · +925 -467

Commits on Dec 2, 2025

parameterize nx, sim_time and output_freq in command line J Franco Ray committed 3 days ago		87bc4d9
Refactor command line argument handling J Franco Ray committed 3 days ago		92aa120
refactor J Franco Ray committed 3 days ago		61bb4bd
Merge branch 'dev' into feat/input-params RaionG18 authored 3 days ago · ✓ 1 / 1	Verified	14ec97e
Merge pull request #23 from prabhkodes/feat/input-params RaionG18 authored 3 days ago · ✓ 1 / 1	Verified	09b80a5
changed slurm file Prabhsharan committed 3 days ago · ✓ 1 / 1		e974070
Merge pull request #25 from prabhkodes/prabh_dev RaionG18 authored 3 days ago · ✓ 1 / 1	Verified	ce0902b
Add OpenACC support for parallel processing in ztend and update flux calculations RaionG18 committed 3 days ago		7781b1f
Refactor Makefile to enhance OpenACC support and improve compiler flag management RaionG18 committed 3 days ago		8f11ec0
Refactor test target in Makefile for improved readability and maintainability RaionG18 committed 3 days ago		ba9956d
Format Makefile for improved readability and consistency RaionG18 committed 3 days ago		21be2b3

Make your life (more) easier

- Compilation errors
  - Properly load modules, sequence matters
  - Set appropriate compiler and library flags
  - May need to use indent for pragmas
- Batch file configuration
  - Dedicate only 1 GPU per rank
  - Use `cpus-per-task` for multithreading
  - Use `boost_qos_dbg` qos for higher priority
- Program is not scaling
  - Check data flow and references before, within and outside the parallel regions

Retrospective

What we did best

- Using GitHub repo, projects and actions
- Branching and merging PRs
- Automated testing
- Documentation

What could be done better

- Overlap communication with computation
- Fancy: include GitHub action to run tests directly on the cluster

Release

Parallelisation Using OpenMPI + OpenMP

Latest

We have parallelised the code using mpi and open mp.
The domain is decomposed using `setup_domain_decomposition` subroutine.
The boundaries are exchanged using MPI in subroutine `exchange_halo_x` since domain is decomposed along x-axis
The mass and energy of the system are gotten from all ranks using MPI Allreduce in `total_mass_energy` subroutine
The computation is parallelised using openmp directives
Added Doxygen and code documentation
Updated slurm scripts to benchmark performance

What's Changed

Parallelisation Using OpenMPI + OpenACC

This release adds support for multi-gpu open acc

- Feature openacc by **@RaionG18** in #32
- added automated test to compare generated output with the reference output by **@formidablefrank** in #26
- Merge MPI Changes into open acc changes by **@prabhkodes** in #34

Contributors

formidablefrank, RaionG18, and prabhkodes

Assets 2

Source code (zip)	2 hours ago
Source code (tar.gz)	2 hours ago

1 You reacted